



Thaneeshga M 251801354-2025-AIDS ▾

T2**Started on** Thursday, 8 October 2026, 10:11 AM**State** Finished**Completed on** Thursday, 8 October 2026, 10:16 AM**Time taken** 4 mins 43 secs**Grade** 10.00 out of 10.00 (100%)

Question 1 | Correct | Mark 10.00 out of 10.00**Playing with Numbers:**

Ram and Sita are playing with numbers by giving puzzles to each other. Now it was Ram term, so he gave Sita a positive integer 'n' and two numbers 1 and 3. He asked her to find the possible ways by which the number n can be represented using 1 and 3. Write any efficient algorithm to find the possible ways.

Example 1:**Input:** 6**Output:** 6**Explanation:** There are 6 ways to 6 represent number with 1 and 3

1+1+1+1+1+1

3+3

1+1+1+3

1+1+3+1

1+3+1+1

3+1+1+1

Input Format

First Line contains the number n

Output Format**Print: The number of possible ways 'n' can be represented using 1 and 3**

Sample Input

6

Sample Output

6

Answer: (penalty regime: 0 %)

```

1  #include <stdio.h>
2
3  int main() {
4      int n;
5      scanf("%d", &n);
6      long long dp[n + 1];
7      dp[0] = 1;
8      for (int i = 1; i <= n; i++) {
9          dp[i] = dp[i - 1];
10         if (i >= 3)
11             dp[i] += dp[i - 3];
12     }
13     printf("%lld", dp[n]);
14     return 0;
15 }
```

	Input	Expected	Got	
✓	6	6	6	✓

	Input	Expected	Got	
✓	25	8641	8641	✓
✓	100	24382819596721629	24382819596721629	✓

Passed all tests! ✓

Correct

Marks for this submission: 10.00/10.00.

[Back to Course](#)

**Thaneeshga M 251801354-2025-AIDS** ▾**T2****Started on** Thursday, 8 October 2026, 10:16 AM**State** Finished**Completed on** Thursday, 8 October 2026, 10:19 AM**Time taken** 2 mins 30 secs**Grade** 10.00 out of 10.00 (100%)

Question 1 | Correct | Mark 10.00 out of 10.00**Playing with Chessboard:**

Ram is given with an $n \times n$ chessboard with each cell with a monetary value. Ram stands at the (0,0), that the position of the top left white rook. He is been given a task to reach the bottom right black rook position ($n-1$, $n-1$) constrained that he needs to reach the position by traveling the maximum monetary path under the condition that he can only travel one step right or one step down the board. Help ram to achieve it by providing an efficient DP algorithm.

Example:**Input**

```
3
1 2 4
2 3 4
8 7 1
```

Output:

```
19
```

Explanation:

Totally there will be 6 paths among that the optimal is

Optimal path value: $1+2+8+7+1=19$

Input Format

First Line contains the integer n

The next n lines contain the $n \times n$ chessboard values

Output Format

Print Maximum monetary value of the path

Answer: (penalty regime: 0 %)

```
1 |
2 | #include <stdio.h>
3 |
4 | int main() {
5 |     int n;
6 |     scanf("%d", &n);
7 |
8 |     int board[n][n];
9 |     int dp[n][n];
10 |
11 |     for (int i = 0; i < n; i++) {
12 |         for (int j = 0; j < n; j++) {
13 |             scanf("%d", &board[i][j]);
14 |         }
15 |     }
16 |
17 |     dp[0][0] = board[0][0];
18 |     for (int j = 1; j < n; j++) {
19 |         dp[0][j] = dp[0][j - 1] + board[0][j];
20 |     }
21 |     for (int i = 1; i < n; i++) {
22 |         dp[i][0] = dp[i - 1][0] + board[i][0];
23 |     }
24 |
25 |     for (int i = 1; i < n; i++) {
26 |         for (int j = 1; j < n; j++) {
27 |             int max = dp[i - 1][j];
28 |
29 |             if (dp[i][j - 1] > max)
30 |                 max = dp[i][j - 1];
31 |
32 |             dp[i][j] = board[i][j] + max;
33 |         }
34 |     }
35 |     printf("%d", dp[n - 1][n - 1]);
36 | }
```

```
37 |      return 0;  
38 |  }
```

	Input	Expected	Got	
✓	3 1 2 4 2 3 4 8 7 1	19	19	✓
✓	3 1 3 1 1 5 1 4 2 1	12	12	✓
✓	4 1 1 3 4 1 5 7 8 2 3 4 6 1 6 9 0	28	28	✓

Passed all tests! ✓

Correct

Marks for this submission: 10.00/10.00.

[Back to Course](#)



**Thaneeshga M 251801354-2025-AIDS** ▾**T2****Started on** Thursday, 8 October 2026, 10:19 AM**State** Finished**Completed on** Thursday, 8 October 2026, 10:21 AM**Time taken** 1 min 48 secs**Marks** 1.00/1.00**Grade** **10.00** out of 10.00 (**100%**)

Question 1 | Correct | Mark 1.00 out of 1.00

Given two strings find the length of the common longest subsequence(need not be contiguous) between the two.

Example:

s1: ggtabe

s2: tgatasb

s1		a	g	g	t	a	b	
s2		g	x	t	x	a	y	b

The length is 4

Solving it using Dynamic Programming

For example:

Input	Result
aab	2
azb	

Answer: (penalty regime: 0 %)

```

1  #include <stdio.h>
2  #include <string.h>
3
4  int main() {
5      char s1[1000], s2[1000];
6
7      scanf("%s", s1);
8      scanf("%s", s2);
9
10     int m = strlen(s1);
11     int n = strlen(s2);
12
13     int dp[m + 1][n + 1];
14     for (int i = 0; i <= m; i++)
15         dp[i][0] = 0;
16
17     for (int j = 0; j <= n; j++)
18         dp[0][j] = 0;
19
20     for (int i = 1; i <= m; i++) {
21         for (int j = 1; j <= n; j++) {
22
23             if (s1[i - 1] == s2[j - 1]) {
24                 dp[i][j] = dp[i - 1][j - 1] + 1;
25             } else {
26                 if (dp[i - 1][j] > dp[i][j - 1])
27                     dp[i][j] = dp[i - 1][j];
28                 else
29                     dp[i][j] = dp[i][j - 1];
30             }
31         }
32     }
33
34     printf("%d", dp[m][n]);
35
36     return 0;
37 }
```

	Input	Expected	Got	
✓	aab azb	2	2	✓
✓	ABCD ABCD	4	4	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)

**Thaneeshga M 251801354-2025-AIDS** ▾**T2****Started on** Thursday, 8 October 2026, 10:21 AM**State** Finished**Completed on** Thursday, 8 October 2026, 10:48 AM**Time taken** 26 mins 38 secs**Marks** 1.00/1.00**Grade** **10.00** out of 10.00 (**100%**)

Question 1 | Correct | Mark 1.00 out of 1.00

Problem statement:

Find the length of the Longest Non-decreasing Subsequence in a given Sequence.

Eg:

Input:9

Sequence:[-1,3,4,5,2,2,2,3]

the subsequence is [-1,2,2,2,2,3]

Output:6

Answer: (penalty regime: 0 %)

```

1  #include <stdio.h>
2
3  int main() {
4      int n;
5      scanf("%d", &n);
6
7      int a[n], dp[n];
8
9      for (int i = 0; i < n; i++) {
10         scanf("%d", &a[i]);
11         dp[i] = 1;
12     }
13     for (int i = 1; i < n; i++) {
14         for (int j = 0; j < i; j++) {
15             if (a[j] <= a[i] && dp[j] + 1 > dp[i]) {
16                 dp[i] = dp[j] + 1;
17             }
18         }
19     }
20
21     int max = dp[0];
22
23     for (int i = 1; i < n; i++) {
24         if (dp[i] > max) {
25             max = dp[i];
26         }
27     }
28
29     printf("%d\n", max);
30
31     return 0;
32 }
```

	Input	Expected	Got	
✓	9 -1 3 4 5 2 2 2 2 3	6	6	✓
✓	7 1 2 2 4 5 7 6	6	6	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)

